Session 6 · Deck 6.A2

Aiyagari II: From a Policy to a Population, and a Price

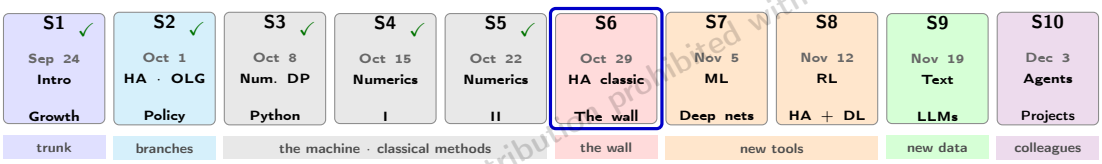
Zhigang Feng

Thursday, October 29, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — Aiyagari: 6.A1 the household, from Bellman to Euler · **6.A2** the population and the price.
Block B: 6.B1 Krusell–Smith · 6.B2 transitions and the life cycle.

© Zhigeng Fang
Redistribution prohibited without permission

Roadmap: Six Parts

6.A1 delivered a saving policy at one price. This deck turns it into a population, and the population into a price.

Part	Topic	The question it answers
1	One Markov chain on the grid	How does a policy move a population, and what do we solve for?
2	Mass that lands between nodes	What happens when a' falls between two nodes?
3	Three ways to the fixed point	Iterate, solve, or take an eigenvector: what does each cost?
4	Why it works, and what to check	Why one answer, and why not simulate households?
5	Clear the market	How do we find the price, and why bisection with a sign check?
6	The equilibrium, and what it says	Is it right, and what does it get right about inequality?

© Zhigang Feng Redistribution prohibited without permission

Idea

Given a policy, the population is the fixed point of a linear map; given the population, the price is the root of one scalar function. Both steps are simple, and each has one trap.

1 · One Markov Chain on the Grid

A household at (a_i, ε_q) saves $a'(a_i, \varepsilon_q)$ and draws ε' from Π. The policy and the chain move every household; the population is where that motion stops.

The operator. Mass $\lambda(a, \varepsilon)$ today becomes, tomorrow,

$$\lambda'(a', \varepsilon') = \sum_{\varepsilon} \sum_{a:\, a'(a,\varepsilon)=a'} \lambda(a, \varepsilon)\, \Pi(\varepsilon, \varepsilon').$$

This is the Markov operator Q of 2.A3. On the grid it is a $1{,}400 \times 1{,}400$ matrix P_g, and one step is a row vector times a matrix: $\lambda' = \lambda P_g$.

The object. The invariant distribution solves

$$\lambda = \lambda P_g, \qquad \sum \lambda = 1.$$

Idea

The policy is fixed now: this is a *linear* problem, unlike 6.A1's.

Route	What it does	Part
iterate	$\lambda_{n+1} = \lambda_n P_g$ until it stops	3
linear solve	$(P_g^\top - I)\lambda = 0$, $\sum \lambda = 1$	3
eigenvector	left eigenvector at eigenvalue 1	3
simulate	a panel of households	4: why not

But first a problem every route shares: $a'(a_i, \varepsilon_q)$ is almost never a node. Where does the household's mass go?

© Zhigang Feng
Redistribution prohibited without permission

Notation as fixed in Session 2: λ the invariant distribution, Q the operator on measures.

Off the Grid: Rounding Moves the Mean, the Lottery Does Not

Rounding a' to the nearest node keeps the mass but moves the mean. Young's lottery splits the household between the two neighbouring nodes and keeps both.

If $a_j \le a' < a_{j+1}$, put

$$\omega = \frac{a' - a_j}{a_{j+1} - a_j},$$

and send the fraction $1 - \omega$ of the household's mass to a_j and ω to a_{j+1}.

Why the mean is exact: $(1-\omega)\,a_j + \omega\,a_{j+1} = a'$. Summed over households, tomorrow's capital $K' = \sum \lambda\, a'$ is exact *given the policy*.

Idea

"Where exactly" becomes "which two nodes, in what proportion": a choice between nodes turns into a probability split between them.

Pitfall: the mean, not the spread

Each split adds a little dispersion: the lottery keeps $\mathbb{E}a'$, not the variance. A coarse grid therefore widens the distribution, which is why 6.B1 pushes its histogram on nodes 0.1 apart.

© Zhigang Feng — Redistribution prohibited without permission

Young, "Solving the incomplete markets model with aggregate uncertainty using the Krusell–Smith algorithm and non-stochastic simulations", *JEDC* 34 (2010).

Lab 6's first exercise, by hand: nodes at 0, 1, 3; three households with masses 0.5, 0.3, 0.2 want $a' = 0.4$, 2.5, 2.9.

mass	a'	cell	ω	**to lower**	**to upper**
0.5	0.4	$[0,1]$	0.40	0.300	0.200
0.3	2.5	$[1,3]$	0.75	0.075	0.225
0.2	2.9	$[1,3]$	0.95	0.010	0.190

	node 0	node 1	node 3	**mean**
lottery	0.300	0.285	0.415	**1.53**
nearest node	0.5	0	0.5	1.50

Ledger `core.lottery_example`; **Lab 6**, **cell A4**.

Example: check the mean

The true mean is $0.5{\cdot}0.4{+}0.3{\cdot}2.5{+}0.2{\cdot}2.9 = 1.53$. The lottery hits it exactly: $1 \cdot 0.285 + 3{\cdot}0.415 = 1.53$. Rounding sends 2.5 to node 3 and misses by 0.03 — every period, in the same direction if the policy is smooth.

© Zhigang Feng · Redistribution prohibited without permission

The Matrix P_g, Entry by Entry

Number the 1,400 states (a_i, ε_q) as one list. A row is today's state, a column tomorrow's; each row has at most 14 nonzeros.

State (a_i, ε_q) is entry $k = 7i + q$. If $a'(a_i, \varepsilon_q)$ lies in $[a_j, a_{j+1})$ with weight ω, then for every $\varepsilon_{q'}$:

$$P_g\big[(i,q),(j,q')\big] = (1-\omega)\,\Pi_{qq'},$$
$$P_g\big[(i,q),(j{+}1,q')\big] = \omega\,\Pi_{qq'}.$$

Two nodes times seven productivity levels: at most 14 nonzeros per row, and each row sums to one — the household goes *somewhere*.

Example: at $r = 2\%$

$1{,}400 \times 1{,}400 = 1{,}960{,}000$ entries, of which 19,488 are nonzero. Row sums equal one to 4×10^{-16}.

$\lambda' = \lambda P_g$ is a vector times a sparse matrix. Store the 19,488 numbers, or — as the code does — never form the matrix at all (next frame).

© Zhigang Feng
Redistribution prohibited without permission

Ledger `core.distribution_2pct; transition_matrix` **in** `s06_ha.py`.

Way 1: Iterate the Operator

Start everyone at the limit and push the mass forward until it stops moving. This is the power method, and it needs no matrix.

1: $\lambda \leftarrow$ all mass at $a_1 = \underline{a}$, split across ε by π ▷ any start
2: $(j, \omega) \leftarrow$ lottery(a') ▷ once: a' is fixed
3: **repeat**
4: for each q: $m \leftarrow \lambda_{\cdot q}$
5: $t \leftarrow$ `bincount`$(j,\, m(1-\omega))+$`bincount`$(j{+}1,\, m\omega)$
6: $\lambda^{\text{new}} \leftarrow \lambda^{\text{new}} + t\,\Pi_q$. ▷ then draw ε'
7: $d \leftarrow \max|\lambda^{\text{new}} - \lambda|$; $\lambda \leftarrow \lambda^{\text{new}}$
8: **until** $d < 10^{-11}$

Measured

At $r = 2\%$: **493** sweeps, 0.05 s. At 6.A2's equilibrium price: 596 sweeps from the limit, **202** from the previous price's λ.

Why hundreds of sweeps

Each sweep shrinks the distance to λ by about the second-largest eigenvalue of P_g: here $|\lambda_2| = 0.963$. Persistent income means slow mixing.

© Zhigang Feng · Redistribution prohibited without permission

Ledger `core.distribution_2pct` (`second_eigenvalue_modulus`), `core.warm_start`; **Lab 6**, **cell A5**.

The invariant λ solves a linear system with one redundant equation. It is also P_g's left eigenvector at eigenvalue 1.

Way 2. $(P_g^\top - I)\lambda = 0$ is 1,400 equations of which only 1,399 are independent: every row of P_g sums to one, so the equations sum to zero. Replace one of them by $\sum \lambda = 1$ and solve once. A dense solve costs order n^3 (4.A2): cheap at 1,400 states, prohibitive at a hundred times more.

Way 3. $\lambda P_g = 1 \cdot \lambda$: the left eigenvector at eigenvalue 1, scaled to sum to one. Way 1 is the power method for exactly this eigenvector.

Measured, $r = 2\%$

Way 2: 0.06 s. Way 3: 0.81 s. Way 2 against Way 1: largest gap 2.5×10^{-10}, the iteration's tolerance. Way 3 against Way 2: 6×10^{-16}.

One fixed point, three algorithms. Choose by size: a dense solve for thousands of states, the sparse iteration beyond.

Ledger `core.distribution_2pct`. **Equivalently $\lambda = Q^\top \lambda$: λ is an eigenvector of $Q^\top$ for the eigenvalue 1.**

© Zhigang Feng Not to be distributed without permission

Existence, Uniqueness, Convergence — in Words

2.A3 proved them from three properties of the operator. Here is why each one holds in this economy.

Property (2.A3)	Guarantees	Why it holds here
bounded assets	a compact state	$\beta(1+r)<1$: wealth has a ceiling (6.A1's net-saving picture)
Feller	existence	the policy is continuous in a; Π is a finite chain
mixing	uniqueness	a run of low ε takes anyone to the limit, a run of high ε takes anyone up: every state communicates with the limit
monotonicity	convergence from any start	a' rises with a, and a higher ε today means a higher ε tomorrow in distribution

© Zhiguo Feng · Redistribution prohibited without permission

Pitfall: 2.A3's two innocent matrices

With $Q_a = I$ every distribution is invariant; with Q_b that swaps two states, iteration from $(1,0)$ never converges. A solver reports an answer in both cases. Part 3's agreement of three routes, and Part 6's checks, are how you notice.

Simulating a panel of households is the obvious alternative. It is noisy — and anything that differentiates through the distribution inherits the noise.

Lab 6, B2: from the equilibrium λ, households follow $a' = \theta_0 + \theta_1 \mathrm{coh}$. How does capital T periods on respond to θ_1? Four answers:

T	autograd	finite diff.	panel, common	panel, indep.
1	6.896	6.896	6.90 ± 0.05	4.8 ± 34.4
20	97.21	97.21	97.15 ± 0.82	89.7 ± 51.7
mass at the limit, $T = 5$				
5	-0.349	-0.349	-0.66 ± 0.18	-0.31 ± 2.79

Reading it. Each entry: how much K (last row: the mass at the limit) after T periods rises if θ_1 rises by one. **Autograd:** the exact derivative through T lottery steps. **Finite diff.:** the same histograms at $\theta_1 \pm 10^{-6}$. **Panel:** 10,000 simulated households at $\theta_1 \pm 0.001$, the gap divided by 0.002; mean $\pm$ s.d. over 20 seeds; *common* reuses one set of draws in both runs, *indep.* draws anew. At $T = 1$ the answer is mean cash on hand.

Idea

The lottery's weights are linear in a': the histogram is smooth in the policy, and autograd through it equals the finite difference to 2×10^{-10}.

The panel

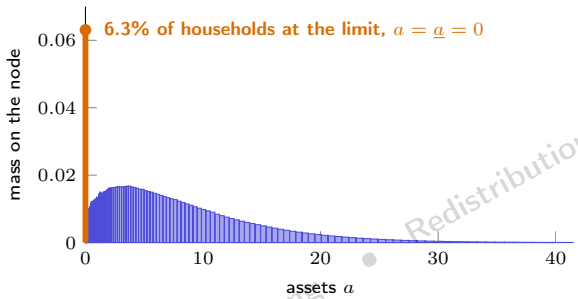
Independent draws bury the derivative in noise. Common draws rescue capital, not the mass at the limit: it moves one household at a time.

Ledger `lottery_grad`; **torch 2.8. This is the measured reason 6.B1 pushes histograms, and Session 8 differentiates through them.**

© Eric R. Scheffler · Redistribution prohibited without permission

The Distribution, Pictured, and Two Checks

The invariant distribution at $r = 2\%$, mass by asset node. Two checks before anyone uses it.



© Zhigang Feng Redistribution prohibited without permission

Reading it

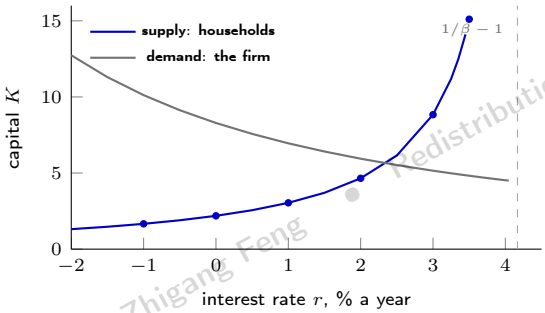
6.3% of households sit at the limit (the orange spike at $a = 0$); mean assets are 4.656 — the capital households supply at $r = 2\%$. Bars widen to the right because the cells do.

Two checks

(1) $\sum \lambda = 1$: here exact to machine precision. (2) Mass on the top node: 0 here. Were it 10^{-3}, the grid, not the economics, would set K: widen it.

Ledger `core.distribution_2pct`, `fig_wealth_nodes_2pct`; **the first 120 of 200 nodes. Lab 6, cell A5.**

Each point of the supply curve is one household solve (6.A1) plus one distribution (Parts 1–4). This is 2.A2's figure, now computed.



Reading it

Supply rises from 1.66 at $r = -1\%$ to 38.6 at 3.95% and has no bound as $r \to 1/\beta - 1$. Demand falls. They cross once.

Each point costs about 0.1 s from scratch. The next frames find the crossing with about twenty of them.

Ledger `core.capital_market, fig_capital_market`; **dots**: **Lab 6's seven points (cell A6)**, **warm-started**. `core.inner_solve_seconds`.

© Zhigang Feng
Redistribution prohibited without permission

Bisection halves any bracket it is given. If the root is not inside, it still stops at an edge and reports a tidy number.

A careless bisection (Lab 6, A7) on $[-2\%, 2\%]$ "converges" to $r = 2.000\%$: the edge of its bracket. Households supply 4.66, the firm demands 5.94 — **the market is 21.6% off**, and nothing in the output says so until the clearing residual is printed.

The rule. Evaluate both ends first, and refuse to start unless $d(r_{\text{lo}}) < 0 < d(r_{\text{hi}})$. Here $d = -64.9$ at $r = -6\%$ and $+105.5$ at 4.1657%: a root is inside.

Which bracket: the economics

Lower end: $r > -\delta$, or capital is free; the code uses $-\delta + 2\%$.

Upper end: the after-tax return must stay below $1/\beta - 1$; above it wealth has no ceiling and supply is infinite. The code uses $(1/\beta - 1)/(1 - \tau_k) - 10^{-5}$.

Ledger `core.careless_bisection`, `core.equilibrium.sign_change`; **4.A1**, "**Bisection: The One That Cannot Fail**".

© Zhigang Feng · Redistribution prohibited without permission

Every evaluation of $d(r)$ costs a full inner solve, so the outer method must need few of them and must never fail.

Why not Newton (4.A1, 4.A2)

$d(r)$ is only piecewise smooth: as r moves, the set of nodes where the limit binds changes discretely, and the lottery's node indices jump. A finite-difference derivative across such a switch is not the derivative of anything, and a Newton step can leave the bracket. Monotonicity of d is not a theorem either.

Bisection needs only a sign: 20 halvings take the bracket from about 10% to 10^{-7}.

Ledger `core.equilibrium`, `core.warm_start`, `core.dense_solve_equilibrium`.

Warm starts

Each new r starts from the previous policy and distribution. At r^*: **142** EGM sweeps instead of 356, **202** distribution sweeps instead of 596.

The whole equilibrium: 20 bisections, 3.4 s. With Way 2's dense solve for λ at every price: 2.2 s.

© Zhigang Feng Redistribution prohibited without permission

$r^* = 2.356\%$, 1.81 points below $1/\beta - 1$: precautionary saving, as a number.

interest rate r^*	2.3559%
patience, $1/\beta - 1$	4.1667%
capital K^*	5.639
wage w	1.186
output Y	1.770
K/Y	3.19
saving rate $\delta K/Y$	25.5%

Idea

Without income risk, $r = 1/\beta - 1$ and $K = 4.43$ (the last row two frames on). With uninsurable risk households save more for bad times: capital is 27% higher and the return 1.81 points lower. That is Aiyagari's (1994) result, measured.

Example: how precise

20 bisections; the final bracket is narrower than 10^{-7}, and the market clears to 4.3×10^{-6} of demand.

Ledger `core.equilibrium`, `core.precautionary`. **Aiyagari**, "**Uninsured idiosyncratic risk and aggregate saving**", ***QJE*** **(1994)**.

© Zhigang Feng
Redistribution prohibited without permission

Five numbers before believing an equilibrium. Only the last is expensive, and it is the one that most often changes an answer.

1. **The market clears:** $|K^s - K^d|/K^d = 4.3 \times 10^{-6}$.
2. **The Euler equation holds between nodes:** mass-weighted mean $10^{-4.95}$; worst $10^{-1.79}$, in the cell with the kink.
3. **The grid is wide enough:** mass on the top node is zero.
4. **Mass is conserved:** $|\sum \lambda - 1| = 1.1 \times 10^{-16}$.
5. **The answer survives refinement:** the table.

n_a	r^*, %	K^*	Gini	Euler
100	2.3436	5.649	0.537	$10^{-4.38}$
200	2.3559	5.639	0.534	$10^{-4.95}$
400	2.3594	5.637	0.534	$10^{-5.61}$
800	2.3602	5.636	0.534	$10^{-6.22}$

From 200 to 800 nodes r^* moves by 0.004 points and the Gini not at all: 200 nodes are enough for these numbers.

Ledger `core.five_checks`, `core.refinement` **(Euler: mass-weighted mean residual).**

© Zhigang Feng · Redistribution prohibited without permission

What It Gets Right, What It Misses — and It Is Not the Calibration

Uninsurable earnings risk and a borrowing limit alone make wealth twice as unequal as earnings. The top is missing, and no parameter here brings it back.

	model	U.S.
Gini, earnings	0.248	
Gini, wealth	0.534	≈ 0.82
top 10% share	34.0%	
top 1% share	5.5%	$\approx \frac{1}{3}$
at the limit	2.9%	

U.S. figures as cited in 1.A2 (SCF).

economy	r^*, %	Gini	at limit	top 1%
persistence 0.60	3.08	0.41	0.5%	3.9%
persistence 0.90	2.36	0.53	2.9%	5.5%
persistence 0.95	2.44	0.60	10.2%	6.7%
risk s.d. 0.30	3.31	0.52	5.2%	5.1%
risk s.d. 0.60	1.39	0.55	2.5%	5.9%
limit -1	2.54	0.61	2.3%	6.0%
limit -2	2.68	0.68	1.8%	6.4%
no risk	4.17	—	—	—

Persistence rows hold the unconditional s.d. of $\log \varepsilon$ fixed. 2.A1 promised this number: persistence moves the constrained share from 0.5% to 10.2%.

Ledger `core.five_checks`, `core.statics`; **"at limit" is the mass at the borrowing limit, which moves with the limit itself.**

© Zhigang Feng
Reproduction prohibited without permission

The Whole Algorithm on One Frame

The three nested problems of 6.A1, with every line now built and measured.

grid, ε, $\Pi \leftarrow$ power grid, Rouwenhorst ▷ 6.A1, Part 1
$r_{\text{lo}} \leftarrow -\delta + 0.02$; $r_{\text{hi}} \leftarrow (1/\beta - 1)/(1 - \tau_k) - 10^{-5}$
evaluate $d(r_{\text{lo}})$, $d(r_{\text{hi}})$; stop unless $d(r_{\text{lo}}) < 0 < d(r_{\text{hi}})$ ▷ Part 5
while $r_{\text{hi}} - r_{\text{lo}} > 10^{-7}$ **do**
    $r \leftarrow (r_{\text{lo}} + r_{\text{hi}})/2$; $(K^d, w) \leftarrow$ the firm at r
    $c, a' \leftarrow$ the household by EGM, warm-started ▷ 6.A1
    $\lambda \leftarrow$ invariant distribution by iteration, warm-started ▷ Parts 1–3
    $d \leftarrow \sum \lambda\, a - K^d$; if $d > 0$: $r_{\text{hi}} \leftarrow r$, else $r_{\text{lo}} \leftarrow r$
end while
report r^*, K^*, w, and the five checks ▷ Part 6

`solve_equilibrium` in `s06_ha.py`, **Lab 6**, **cell A8**.

Ten lines, three fixed points. Lines 6 and 7 are the expensive ones; the warm starts are why they are cheap after the first price.

© Zhiyang Feng • Redistribution prohibited without permission

How It Works (1): The Lottery, One Push, and the Matrix

```python
def lottery(ap, grid):
    """Young (2010): split the mass at a' between the two neighbouring nodes."""
    a1 = np.clip(ap, grid[0], grid[-1])
    j = np.clip(np.searchsorted(grid, a1, side="right") - 1, 0, grid.size - 2)
    return j, (a1 - grid[j]) / (grid[j + 1] - grid[j])
```

```python
def push_forward(lam, j, w_hi, Pi):
    """One application of the operator on measures: lambda_t -> lambda_{t+1}."""
    na, ne = lam.shape
    out = np.zeros_like(lam)
    for q in range(ne):
        m = lam[:, q]
        tmp = (np.bincount(j[:, q], weights=m * (1.0 - w_hi[:, q]), minlength=na)
               + np.bincount(j[:, q] + 1, weights=m * w_hi[:, q], minlength=na))
        out += np.outer(tmp, Pi[q, :])
    return out
```

```python
def transition_matrix(j, w_hi, Pi):
    """The same operator as a dense matrix over (a, e) pairs, row-stochastic."""
    na, ne = j.shape
    P = np.zeros((na * ne, na * ne))
    rows = np.arange(na)
    for q in range(ne):
        for qn in range(ne):
            lo, hi = j[:, q] * ne + qn, (j[:, q] + 1) * ne + qn    # the two nodes
            np.add.at(P, (rows * ne + q, lo), (1.0 - w_hi[:, q]) * Pi[q, qn])
            np.add.at(P, (rows * ne + q, hi), w_hi[:, q] * Pi[q, qn])
    return P
```

© Zhiyang Feng · Redistribution prohibited without permission

How It Works (2): The Invariant Distribution, Two Ways

```python
def stationary_distribution(ap, grid, Pi, pi, method="solve", lam_init=None,
                            tol=1e-11, max_iter=200000):
    """Invariant distribution over (a, e): by a linear solve, or by iteration."""
    na, ne = ap.shape
    j, w_hi = lottery(ap, grid)
    if method == "solve":
        A = transition_matrix(j, w_hi, Pi).T - np.eye(na * ne)
        A[-1, :] = 1.0                                  # replace one equation by sum = 1
        b = np.zeros(na * ne); b[-1] = 1.0
        lam = np.maximum(np.linalg.solve(A, b), 0.0).reshape(na, ne)
        return lam / lam.sum(), 1, 0.0
    lam = np.zeros((na, ne)); lam[0, :] = pi          # everyone starts at the limit
    if lam_init is not None:
        lam = lam_init.copy()
    diff, it = np.inf, 0
    for it in range(1, max_iter + 1):
        new = push_forward(lam, j, w_hi, Pi)
        diff = float(np.abs(new - lam).max())
        lam = new
        if diff < tol:
            break
    return lam / lam.sum(), it, diff
```

© Zhirang Feng • Redistribution prohibited without permission

`method="solve"` is Way 2, the default branch is Way 1. Lab 6, cell A5 times both.

How It Works (3): One Point of the Supply Curve, and the Bisection

```python
def capital_supply(r, p: Params, grid, e, Pi, pi, c_init=None, dist="iterate",
                   lam_init=None):
    """Households' capital at a pre-tax return r; the tax is rebated lump-sum."""
    kl, w = firm(r, p)
    r_net, transfer = (1.0 - p.tau_k) * r, p.tau_k * r * kl
    ap, c, it, diff = solve_household(r_net, w, transfer, grid, e, Pi, p,
                                      c_init=c_init)
    lam, it_d, _ = stationary_distribution(ap, grid, Pi, pi, method=dist,
                                           lam_init=lam_init)
    Ks = float((lam.sum(axis=1) * grid).sum())
    return dict(r=r, r_net=r_net, w=w, transfer=transfer, K_demand=kl, K_supply=Ks,
                ap=ap, c=c, lam=lam, egm_sweeps=it, dist_sweeps=it_d)
```

```python
    lo = capital_supply(r_lo, p, grid, e, Pi, pi, dist=dist)
    hi = capital_supply(r_hi, p, grid, e, Pi, pi, dist=dist)
    d_lo, d_hi = lo["K_supply"] - lo["K_demand"], hi["K_supply"] - hi["K_demand"]
    if not (d_lo < 0.0 < d_hi):                          # no sign change: refuse to run
        raise RuntimeError(f"no sign change on [{r_lo:.5f}, {r_hi:.5f}]: "
                           f"excess supply {d_lo:.4f}, {d_hi:.4f}")
    sol, n = hi, 0
    while r_hi - r_lo > tol:
        n += 1
        sol = capital_supply(0.5 * (r_lo + r_hi), p, grid, e, Pi, pi,
                             c_init=sol["c"], dist=dist, lam_init=sol["lam"])
        d = sol["K_supply"] - sol["K_demand"]
        if verbose:
            print(f"   {n:2d}  r = {sol['r']:.6f}   excess supply = {d:+.6f}")
        if d > 0:
            r_hi = sol["r"]
        else:
            r_lo = sol["r"]
```

© Zhegang Shi. Redistribution prohibited without permission

Takeaways

- **Part 1.** Given the policy, the population is the fixed point of a linear map: $\lambda = \lambda P_g$ on 1,400 states.
- **Part 2.** Young's lottery splits a household between the two nodes around a' and keeps the mean exact; rounding does not.
- **Part 3.** Iterate (493 sweeps, the speed set by $|\lambda_2| = 0.963$), solve once, or take the eigenvector: the same λ to 10^{-10}.
- **Part 4.** One answer exists and iteration finds it from anywhere (2.A3). A simulated panel buries derivatives in noise; the histogram gives them to ten digits.
- **Part 5.** The price comes from bisection that first proves a sign change. Without the check, a bracket ending at 2% "converges" with the market 21.6% off.
- **Part 6.** $r^* = 2.356\%$, 1.81 points below $1/\beta - 1$; five checks pass. Wealth is twice as unequal as earnings, and no parameter here produces the top of the distribution.

© Zhigang Feng · Redistribution prohibited without permission

Bridge: Now Let the Distribution Move

Block A solved an economy in which nothing aggregate ever happens: the distribution sits still, so prices are numbers.

Block B takes that away twice. First an aggregate shock that keeps coming: prices depend on the whole distribution, which becomes a state households must forecast. Then a single surprise — a tax reform — after which the distribution travels from one steady state to another.

Next: 6.B1 and 6.B2

Krusell–Smith: the household of 6.A1 with two more states, solved by value function iteration and by EGM. Then transitions after an MIT shock, and the life cycle.

Homework 2, C1 is this deck's loop on the quarterly calibration of Homework 1.

© Zhigang Feng · Redistribution prohibited without permission